Stacks and queues — everything written down

Updated 23 Sep 2026.

The top of a stack is the head. Push and pop only touch index 0. The front of a queue is the head. Enqueue goes at the back. Dequeue takes the front. Printing the linked list prints top-to-bottom, or front-to-back.

Reverse a queue is written twice. Same loop. The recursive reverse is the other way. Same job.

How a stack is built — LIFO

On a linked list	push = insert at index 0. pop = remove index 0. peek = head.data.
On its own top pointer	new node points at the old top, then top moves to the new node.

push

```
new_node = Node(data)
new_node.next = top
top = new_node
size += 1
```

pop

```
if is_empty():
    raise IndexError ("pop at empty stack")
popped_node = top
top = top.next
size -= 1
return popped_node
```

peek

```
if is_empty():
    raise IndexError ("pop at empty stack")
return top.data
```

is_empty

```
return top is None or return size == 0
```

How a queue is built — FIFO

On a linked list	enqueue = insert at size (the back). dequeue = remove index 0.

Front and rear	enqueue links onto tail. dequeue moves head. there is self.head and self.tail

enqueue

```
new_node = Node(data)
if is_empty():
    head = new_node
else:
    tail.next = new_node
tail = new_node
size += 1
```

dequeue

```
if is_empty():
    raise IndexError
data = head.data
head = head.next
if head is None:
    tail = None
size -= 1
return data
```

empty queue: return head is None. getFront is head.data

Only the top — you never walk the stack

Remove until	pop while the top is not the value. Stop when it is. The value stays. If the value is never the top, the stack ends empty.

```
while not stack.is_empty():
    if stack.peek() != value:
        stack.pop()
    else:
        break
```

When to use

The value you care about is the top, or you only throw away what sits above a value. You do not need the items you pop. They are gone.

Empty into the other one, then put it back — a stack flips, a queue keeps order

Reverse a stack	pop everything into a queue. Dequeue it back onto the stack. Old bottom becomes the new top.
Reverse a queue	dequeue everything onto a stack. Pop it back onto the queue. Old rear becomes the new front.
Reverse a queue, recursive	dequeue one, reverse the rest, enqueue that one. First one out is last one back in.

Reverse a stack

top-to-bottom 1 2 3 becomes 3 2 1

```
q = Queue()
while not stack.isEmpty():
    q.enqueue(stack.pop())
while not q.isEmpty():
    stack.push(q.dequeue())
```

Reverse a queue

front-to-back 10 20 30 becomes 30 20 10

```
stack = Stack()
while not queue.isEmpty():
    stack.push(queue.dequeue())
while not stack.is_empty():
    queue.enqueue(stack.pop())
```

Reverse a queue, recursive

1 2 3

- take 1, reverse 2 3, put 1 at the back
- take 2, reverse 3, put 2 at the back
- take 3, queue empty, put 3 back
- then 2, then 1

ends as 3 2 1

```
if queue.is_empty():
    return
item = queue.dequeue()
```

```
recursive_reverse(queue)
queue.enqueue(item)
```

The written function calls self.recursive_reverse and returns [] on the empty case. It is a plain function. No self. The empty case just returns.

When to use

You need the whole stack or the whole queue in the opposite order. Stack into a queue, then back: the stack is reversed. Queue onto a stack, then back: the queue is reversed. Queue into queue, and back: no change. Stack into Stack and back: no change. Same queue reverse with no second structure: take one off, reverse the rest, put that one at the back.

Flip a piece, and also rotate — the reverse, but only on part of the queue

Reverse the first k	dequeue k onto a stack, enqueue them back, then rotate the leftover. Do nothing if empty, k <= 0, or k is bigger than the size.
Interleave a queue	rotate the first half to the back. Park the second half on a stack. Then dequeue one and pop one, repeating. Odd length, or empty: leave the queue as it is. 1 2 3 4 5 6 becomes 1 6 2 5 3 4.

Reverse the first k

1 2 3 4 5, k = 3

- park 1 2 3. stack top is 3. queue is 4 5
- enqueue the pops: 4 5 3 2 1
- rotate the 2 leftovers: 3 2 1 4 5

```
# always keep if empty thing is sent or the index is out of range
if queue.isEmpty() or k <= 0 or k > queue.size():
    return queue

stack = Stack()
for _ in range(k):
    stack.push(queue.dequeue())

while not stack.is_empty():
    queue.enqueue(stack.pop())

for _ in range(size - k):       # size is still the original size
    queue.enqueue(queue.dequeue())
```

Interleave a queue

half = size // 2

1. rotate the first half to the back

dequeue and enqueue, half times

1 2 3 4 5 6 → 4 5 6 1 2 3

2. park the second half (now at the front) on a stack

the stack reverses it. top is 6. queue left: 1 2 3

3. half times: dequeue one and enqueue it, then pop one and enqueue it

1 then 6, 2 then 5, 3 then 4

1 6 2 5 3 4

```
for _ in range(half):
    q.enqueue(q.dequeue())

for _ in range(half):
    s.push(q.dequeue())

for _ in range(half):
    q.enqueue(q.dequeue())
    q.enqueue(s.pop())
```

When to use

The whole queue should not be reversed. Only a front piece flips, or the two halves have to be woven. Flip the first k, then rotate the leftover to the back. Interleave: rotate the first half back, park the second half on a stack, then one from the queue and one from the stack.

Temp stack, same order comes back — out and back flips twice

Delete middle of a stack	pop size // 2 into a temp stack. Pop once more and throw it. Push the temp back.
Sort a stack	pop one value into a temp that stays sorted, largest on top. If it is smaller than the temp's top, push those bigger ones back until it fits. Smallest ends on top.

Delete middle of a stack

For top-to-bottom A B C D E (A is the top), size 5, depth 2. Park A, B. Throw C. Push B, A back. Left: A B D E. Even size does the same count. Size 4, depth 2: park two, throw the next one.

```
if s.isEmpty():
    return
```

```
temp = Stack()
depth = s.getSize() // 2
for _ in range(depth):
    temp.push(s.pop())

s.pop()                          # the middle, thrown away

while not temp.isEmpty():
    s.push(temp.pop())
```

Sort a stack (smallest on top)

```
temp = Stack()

while not stack.isEmpty():
    data = stack.pop()
    while not temp.isEmpty() and data < temp.peek():
        stack.push(temp.pop())
    temp.push(data)

# send everything back with the smallest at the top
while not temp.is_empty():
    stack.push(temp.pop())
```

trace, top-to-bottom 3 1 2

- pop 3, temp empty, temp = 3
- pop 1, 1 < 3, so 3 goes back to the original, temp = 1
- pop 3, 3 is not < 1, temp = 3 1
- pop 2, 2 < 3, so 3 goes back, temp = 2 1, then 3 is not < 2, temp = 3 2 1
- push temp back. top-to-bottom 1 2 3

When to use

You have to reach something under the top, and the items you moved must come back in the same order. Delete the middle: park size // 2, throw the next pop, push the temp back. Sort: a second stack that stays sorted. If the new value is smaller than the temp's top, push those bigger ones back until it fits.

Read the string once — the structure remembers what you still owe

Balanced brackets	one stack. Push an opener. On a closer, the top must be its match. Empty at the end means balanced. A closer on an empty stack is not.
Word palindrome	stack and queue. Push and enqueue each letter. Then pop against dequeue. Lowercase first. Skip spaces and punctuation. Only letters and digits.

Balanced brackets

```
s = Stack()
matcher = {")": "(", "]": "[", "}": "{"}  # dictionary with saved matches to do

for i in expression:
    if i in matcher.values():
        s.push(i)
    else:
        if (s.peek() != matcher[i]):
            return False
        else:
            s.pop()

return s.isEmpty()  # in case got extra brackets left that never closed
```

- () — push (, then) matches, pop. empty. balanced
- [()] — [then (then pop (on) then pop [on]. balanced
- [({})] — the closers do not match the top. not balanced

The samples are only brackets. Another character is not in matcher.

Word palindrome

```
q = Queue()
s = Stack()

for i in word:
    i = i.lower()
    # alnum means character or number, no spaces
    if i.isalnum():
        q.enqueue(i)
        s.push(i)

while not s.is_empty():
    if s.pop() != q.dequeue():
        return False
return True
```

"A man a plan a canal Panama" → amanaplanacanalpanama, same backwards. It is written isalum(). The method is isalnum().

When to use

You read a string once, and you have to remember what is still unmatched or still to compare. Brackets: push each opener. A closer must be the match of the top. Empty at the end means balanced. A word the same backwards: push and enqueue the letters, then pop against dequeue.

Which technique when?

Only the top, stop at a value?	pop while peek is not it. the value stays
Whole stack or queue backwards?	empty into the other one, then put it back
Queue backwards, no second structure?	dequeue one, reverse the rest, enqueue that one
Only the first k backwards?	those k go through a stack, then rotate the rest
Weave the two halves?	rotate the first half back, park the second on a stack, then one and one
Reach under the top, same order back?	temp stack. park, do the job, push back
Stack in sorted order?	temp stack. shuffle until the temp's top can take the value
Brackets match?	one stack. push opener, closer must match the top
Word the same backwards?	stack and queue, letters only, pop against dequeue